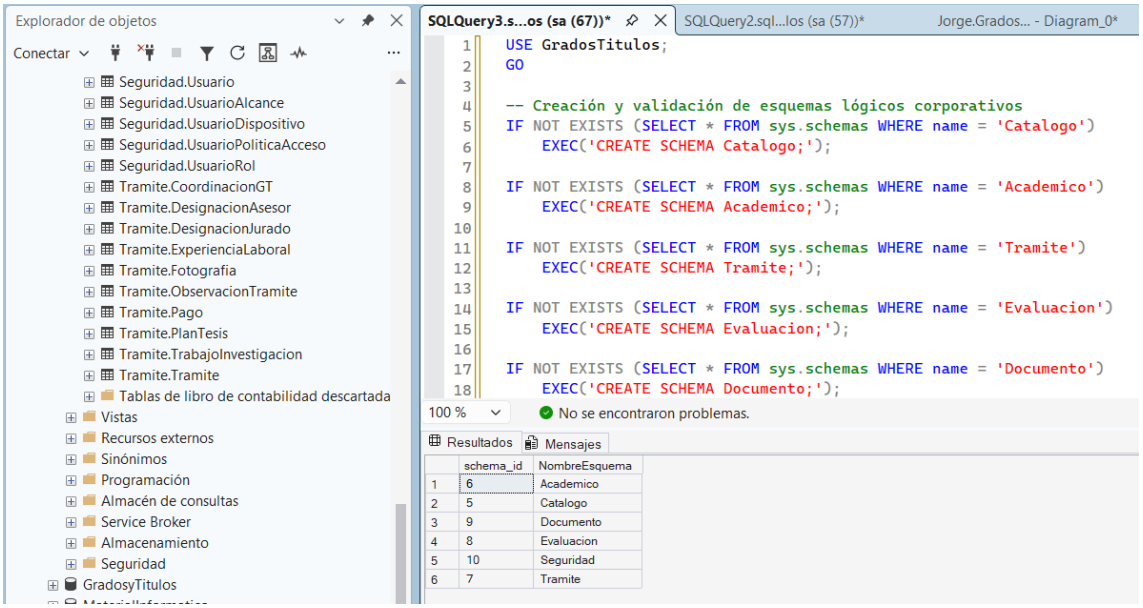
GUÍA DE LABORATORIO N° 01:

CONSOLIDACIÓN Y VERIFICACIÓN DE ESQUEMAS LÓGICOS FUNCIONALES



Respuesta Teórico-Práctica

1. Sustento Teórico: La separación de entidades en esquemas lógicos (Catalogo, Académico, Tramite, Evaluación, Documento, Seguridad) evita agrupar todos los objetos en el esquema por defecto (dbo). Esto mejora directamente el **Control de Acceso Basado en Roles (RBAC)** y el **principio de mínimo privilegio** porque permite asignar o denegar permisos a nivel de contenedor (SCHEMA) en lugar de hacerlo tabla por tabla. Así, cada rol corporativo obtiene únicamente los privilegios estrictamente necesarios para operar en su área funcional, manteniendo aislados y protegidos los módulos críticos del resto de la organización.

2. Demostración en Código SQL Server: El siguiente script sobre la base de datos GradosTitulos demuestra cómo crear roles específicos por área y aplicar el principio de mínimo privilegio usando los esquemas creados en el Laboratorio N° 01:

```sql
USE GradosTitulos;
GO

-- ============================================================================
-- PASO 1: Crear los Roles Corporativos según las funciones del negocio (RBAC)
-- ============================================================================
IF DATABASE_PRINCIPAL_ID('RolMesaDePartes') IS NULL
    CREATE ROLE RolMesaDePartes;

IF DATABASE_PRINCIPAL_ID('RolComiteEvaluador') IS NULL
    CREATE ROLE RolComiteEvaluador;

IF DATABASE_PRINCIPAL_ID('RolAuditorSeguridad') IS NULL
    CREATE ROLE RolAuditorSeguridad;
GO

-- ============================================================================
-- PASO 2: Aplicar el Principio de Mínimo Privilegio por Esquema Lógico
-- ============================================================================

-- A) El rol de Mesa de Partes solo puede leer el Catálogo y registrar/consultar
GRANT SELECT ON SCHEMA::Catalogo TO RolMesaDePartes;
GRANT SELECT, INSERT, UPDATE ON SCHEMA::Tramite TO RolMesaDePartes;

-- B) El Comité Evaluador solo puede leer datos Académicos y gestionar Evaluacior
GRANT SELECT ON SCHEMA::Academico TO RolComiteEvaluador;
GRANT SELECT, INSERT, UPDATE ON SCHEMA::Evaluacion TO RolComiteEvaluador;

-- C) Denegar explícitamente el acceso al esquema Seguridad para roles operativos
DENY SELECT, INSERT, UPDATE, DELETE ON SCHEMA::Seguridad TO RolMesaDePartes;
DENY SELECT, INSERT, UPDATE, DELETE ON SCHEMA::Seguridad TO RolComiteEvaluador;
```

```sql
-- D) Solo el rol de Auditoría puede leer el esquema Seguridad
GRANT SELECT ON SCHEMA::Seguridad TO RolAuditorSeguridad;
GO

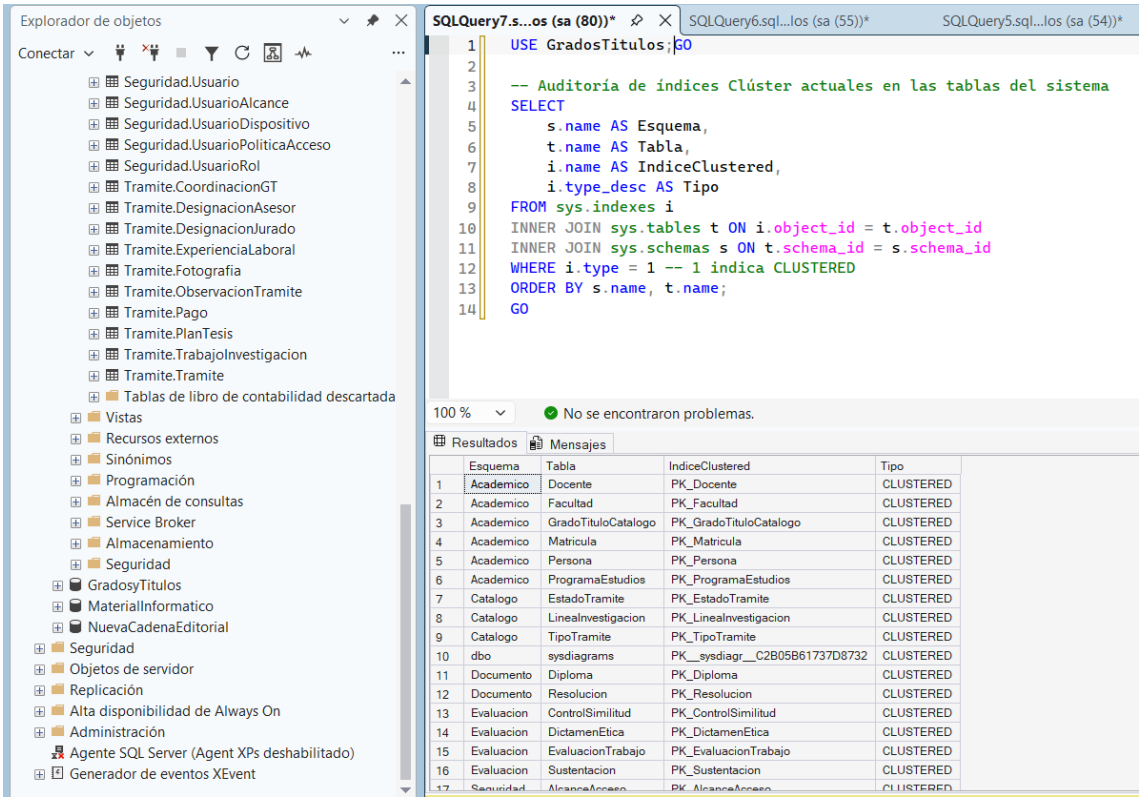
-- ============================================================================
-- PASO 3: Consulta de verificación (Para tu captura de pantalla)
-- Muestra cómo quedaron asignados los permisos por esquema y rol
-- ============================================================================
SELECT
    dp.name AS RolCorporativo,
    perm.state_desc AS Accion,
    perm.permission_name AS Permiso,
    s.name AS EsquemaProtegido
FROM sys.database_permissions perm
INNER JOIN sys.schemas s ON perm.major_id = s.schema_id
INNER JOIN sys.database_principals dp ON perm.grantee_principal_id = dp.principal
WHERE perm.class_desc = 'SCHEMA'
  AND dp.name IN ('RolMesaDePartes', 'RolComiteEvaluador', 'RolAuditorSeguridad')
ORDER BY dp.name, s.name;
GO
```

Resultados | Mensajes

	RolCorporativo	Accion	Permiso	EsquemaProtegido
1	RolAuditorSeguridad	GRANT	SELECT	Seguridad
2	RolComiteEvaluador	GRANT	SELECT	Academico
3	RolComiteEvaluador	GRANT	INSERT	Evaluacion
4	RolComiteEvaluador	GRANT	SELECT	Evaluacion
5	RolComiteEvaluador	GRANT	UPDATE	Evaluacion
6	RolComiteEvaluador	DENY	DELETE	Seguridad
7	RolComiteEvaluador	DENY	INSERT	Seguridad
8	RolComiteEvaluador	DENY	SELECT	Seguridad
9	RolComiteEvaluador	DENY	UPDATE	Seguridad
10	RolMesaDePartes	GRANT	SELECT	Catalogo
11	RolMesaDePartes	DENY	DELETE	Seguridad
12	RolMesaDePartes	DENY	INSERT	Seguridad
13	RolMesaDePartes	DENY	SELECT	Seguridad
14	RolMesaDePartes	DENY	UPDATE	Seguridad
15	RolMesaDePartes	GRANT	INSERT	Tramite
16	RolMesaDePartes	GRANT	SELECT	Tramite
17	RolMesaDePartes	GRANT	UPDATE	Tramite

GUIA DE LABORATORIO N• 02

IMPLEMENTACION DE INDICES CLÚSTER ESTRATÉGICOS EN TABLAS PRINCIPALES



4. Cuestionario de Laboratorio

Explique por qué una tabla de base de datos relacional en SQL Server solo puede tener un único índice Clúster.

1. Sustento Teórico

- **El nivel hoja contiene los datos reales:** En la estructura de árbol B (*B-tree*) de un índice Clúster, el nivel hoja (*leaf level*) no guarda simples punteros o referencias, sino que contiene los propios registros de la tabla. Es decir, el índice Clúster **es** la tabla misma ordenada por la clave elegida.
- **Imposibilidad de doble ordenamiento físico:** Dado que las filas de una misma tabla solo pueden almacenarse siguiendo una única secuencia física a la vez dentro de las páginas de datos, una tabla no puede mantener simultáneamente dos organizaciones físicas distintas de sus mismas filas.

- **Alternativa para otros ordenamientos:** Si se necesita buscar u ordenar rápidamente por otras columnas adicionales (por ejemplo, por DNI o FechaInicio cuando el índice Clúster ya está en el ID), se deben crear índices **No Clúster** (*Nonclustered*), los cuales mantienen estructuras separadas que apuntan hacia la clave del índice Clúster.

2. Demostración en Código SQL Server

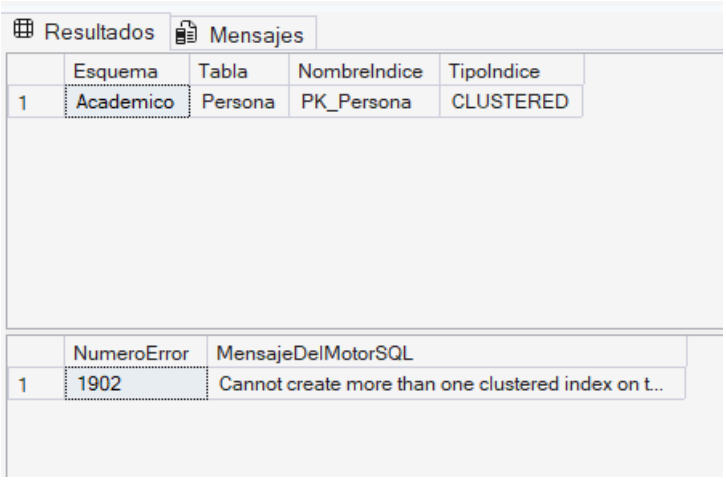
Este script demuestra en tres pasos cómo se crea el único índice Clúster permitido, cómo el motor de SQL Server rechaza automáticamente un segundo índice Clúster sobre la misma tabla (usando un bloque TRY...CATCH para capturar el error oficial en pantalla) y cómo la solución correcta es usar un índice NONCLUSTERED:

```
USE GradosTitulos;
GO

-- ========================================================================
-- PASO 1: Verificar que la tabla Academico.Persona ya tiene 1 Índice Clúster
-- (Creado en su clave primaria IdPersona)
-- ========================================================================
SELECT
    s.name AS Esquema,
    t.name AS Tabla,
    i.name AS NombreIndice,
    i.type_desc AS TipoIndice
FROM sys.indexes i
INNER JOIN sys.tables t ON i.object_id = t.object_id
INNER JOIN sys.schemas s ON t.schema_id = s.schema_id
WHERE s.name = 'Academico'
  AND t.name = 'Persona'
  AND i.type = 1; -- 1 = CLUSTERED
GO

-- ========================================================================
-- PASO 2: Demostración del límite físico (Intentar crear un 2do Índice Clúster
-- SQL Server bloqueará la operación porque las filas ya están ordenadas por Id
-- ========================================================================
BEGIN TRY
    CREATE CLUSTERED INDEX IX_Persona_SegundoClustered
    ON Academico.Persona (DNI);
END TRY
BEGIN CATCH
    SELECT
        ERROR_NUMBER() AS NumeroError,
        ERROR_MESSAGE() AS MensajeDelMotorSQL;
END CATCH;
GO

-- ========================================================================
-- PASO 3: Solución correcta cuando ya existe un Índice Clúster
-- Para ordenar por una segunda columna (DNI), se usa un índice NONCLUSTERED
-- ========================================================================
IF NOT EXISTS (
    SELECT * FROM sys.indexes
    WHERE name = 'IX_Persona_DNI_Demo'
      AND object_id = OBJECT_ID('Academico.Persona')
)
BEGIN
    CREATE NONCLUSTERED INDEX IX_Persona_DNI_Demo
    ON Academico.Persona (DNI);
END
GO
```

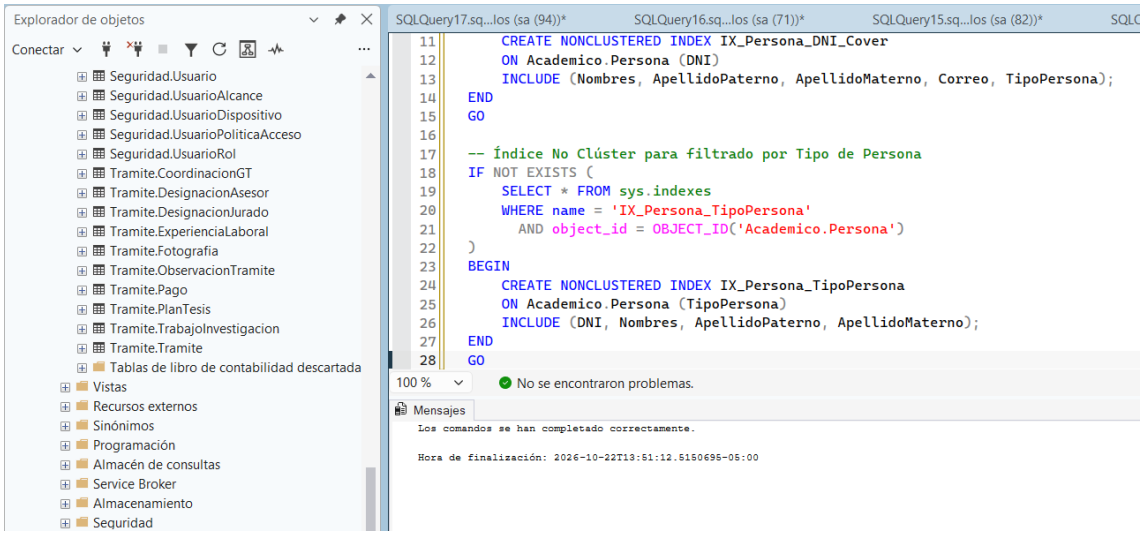

Resultados Mensajes

	Esquema	Tabla	NombreIndice	TipoIndice
1	Academico	Persona	PK_Persona	CLUSTERED

	NumeroError	MensajeDelMotorSQL
1	1902	Cannot create more than one clustered index on t...

GUÍA DE LABORATORIO Nº 03:

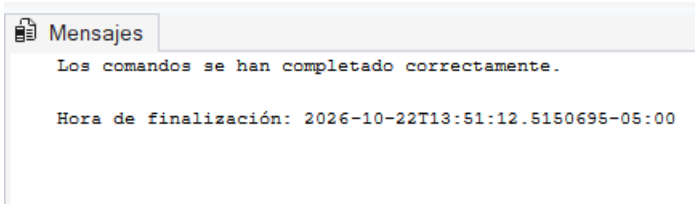
CREACIÓN DE ÍNDICES NO CLÚSTER CON COLUMNAS INCLUIDAS




```sql
USE GradosTitulos;
GO

-- Índice No Clúster con cobertura para búsquedas por DNI
IF NOT EXISTS (
    SELECT * FROM sys.indexes
    WHERE name = 'IX_Persona_DNI_Cover'
      AND object_id = OBJECT_ID('Academico.Persona')
)
BEGIN
    CREATE NONCLUSTERED INDEX IX_Persona_DNI_Cover
    ON Academico.Persona (DNI)
    INCLUDE (Nombres, ApellidoPaterno, ApellidoMaterno, Correo, TipoPersona);
END
GO

-- Índice No Clúster para filtrado por Tipo de Persona
IF NOT EXISTS (
    SELECT * FROM sys.indexes
    WHERE name = 'IX_Persona_TipoPersona'
      AND object_id = OBJECT_ID('Academico.Persona')
)
BEGIN
    CREATE NONCLUSTERED INDEX IX_Persona_TipoPersona
    ON Academico.Persona (TipoPersona)
    INCLUDE (DNI, Nombres, ApellidoPaterno, ApellidoMaterno);
END
GO
```



¿Por qué no aparece una tabla de resultados?

El mensaje *"Los comandos se han completado correctamente"* aparece porque la instrucción que ejecutaste (CREATE NONCLUSTERED INDEX) es un comando de **creación de estructura (DDL)**, no una consulta de lectura de datos.

- Aunque en la línea 6 hay un SELECT * FROM sys.indexes, ese SELECT está dentro de un **IF NOT EXISTS (...)**.
- Su único trabajo ahí es verificar internamente si el índice IX_Persona_DNI_Cover ya existe en la tabla Academico.Persona para evitar que te salga un error de duplicado si ejecutas el script dos veces. No está diseñado para mostrarte filas en pantalla.

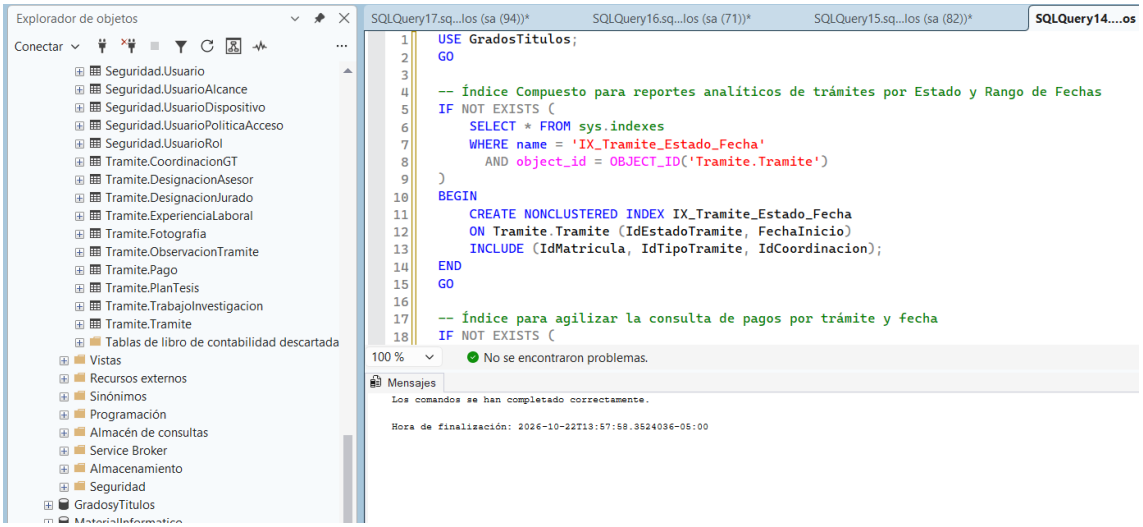
4. Pregunta de Validación

¿Qué diferencia de rendimiento existe entre una consulta que utiliza un índice cubierto con INCLUDE frente a otra que requiere un operador de *Key Lookup*?

"Una consulta que utiliza un **índice cubierto con INCLUDE** presenta un rendimiento muy superior porque obtiene todas las columnas solicitadas directamente desde el nivel hoja del índice mediante un único **Index Seek**, minimizando las lecturas lógicas (I/O). Por el contrario, una consulta que requiere un **Key Lookup** sufre una penalización de rendimiento porque, tras buscar en el índice no clúster, debe realizar lecturas aleatorias adicionales hacia la tabla base (índice clúster) por cada fila recuperada para obtener las columnas faltantes, multiplicando el costo de lectura y el uso de CPU."

GUÍA DE LABORATORIO N° 04:

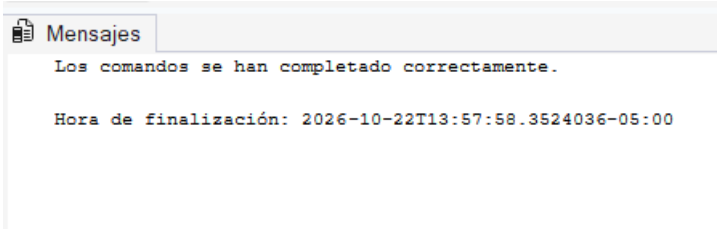
INDEXACIÓN COMPUESTA PARA CONSULTAS ANALÍTICAS DE TRÁMITES




```sql
USE GradosTitulos;
GO

-- Índice Compuesto para reportes analíticos de trámites por Estado y Rango de Fe
IF NOT EXISTS (
    SELECT * FROM sys.indexes
    WHERE name = 'IX_Tramite_Estado_Fecha'
      AND object_id = OBJECT_ID('Tramite.Tramite')
)
BEGIN
    CREATE NONCLUSTERED INDEX IX_Tramite_Estado_Fecha
    ON Tramite.Tramite (IdEstadoTramite, FechaInicio)
    INCLUDE (IdMatricula, IdTipoTramite, IdCoordinacion);
END
GO

-- Índice para agilizar la consulta de pagos por trámite y fecha
IF NOT EXISTS (
    SELECT * FROM sys.indexes
    WHERE name = 'IX_Pago_Tramite_Fecha'
      AND object_id = OBJECT_ID('Tramite.Pago')
)
BEGIN
    CREATE NONCLUSTERED INDEX IX_Pago_Tramite_Fecha
    ON Tramite.Pago (IdTramite, FechaPago)
    INCLUDE (Monto, NroRecibo, TipoPago);
END
GO
```



No se visualizan tablas ni filas de información porque el código ejecutado no es una consulta de recuperación de datos, sino una sentencia de definición para crear un índice (CREATE NONCLUSTERED INDEX) y optimizar el rendimiento de la base de datos.

A continuación, se detallan las dos razones técnicas por las cuales el motor únicamente devuelve el mensaje *"Los comandos se han completado correctamente."*:

1. CREATE INDEX modifica la estructura, no devuelve registros

La instrucción principal en las líneas 11 a 13 (CREATE NONCLUSTERED INDEX IX_Tramite_Estado_Fecha ON Tramite.Tramite...) indica a SQL Server que construya una estructura de búsqueda en segundo plano para la tabla Tramite.Tramite. En SQL Server, cuando se ejecutan comandos de creación o modificación de objetos (como tablas, vistas o índices), el sistema no genera una cuadrícula de resultados; únicamente confirma mediante este mensaje de texto que la estructura fue creada con éxito.

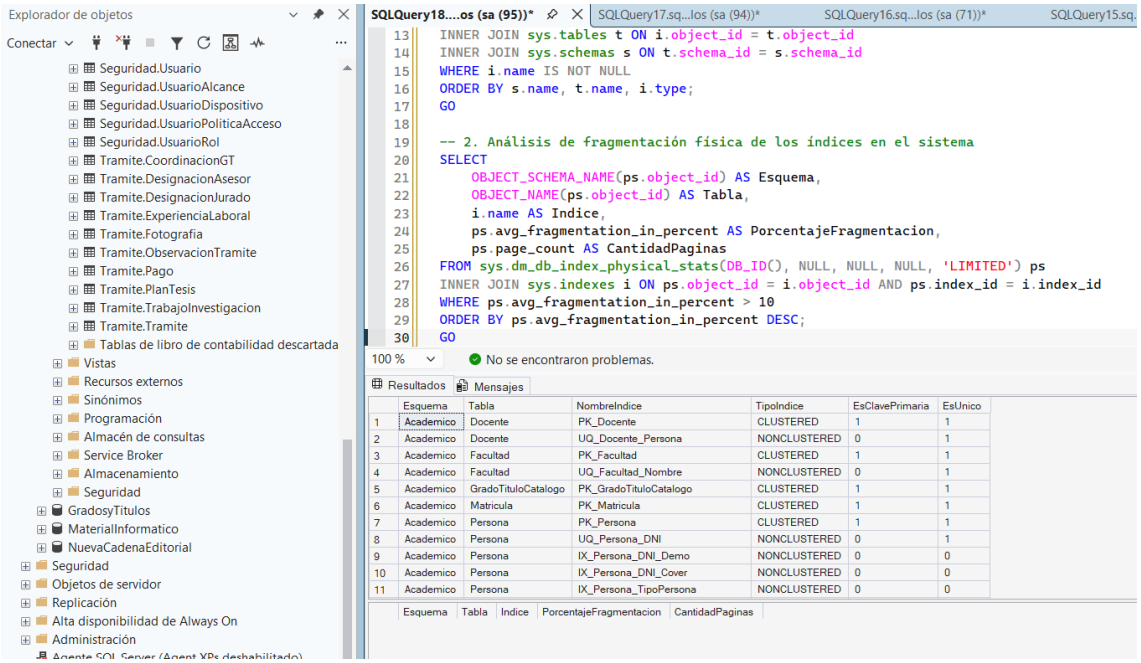
2. La instrucción SELECT * está encapsulada dentro de un condicional IF NOT EXISTS

Aunque en la línea 6 figura la consulta SELECT * FROM sys.indexes, esta se encuentra dentro del paréntesis de la condición **IF NOT EXISTS (...)** (líneas 5 a 9):

- **Funcionamiento interno:** SQL Server ejecuta ese SELECT de manera interna (sin proyectarlo en pantalla) únicamente para validar si el índice ya se encuentra registrado en el sistema.
- **Ejecución del bloque:** Al comprobar que el índice no existía previamente, el motor procede a ejecutar el bloque BEGIN ... END para realizar su creación.

GUÍA DE LABORATORIO N° 05:

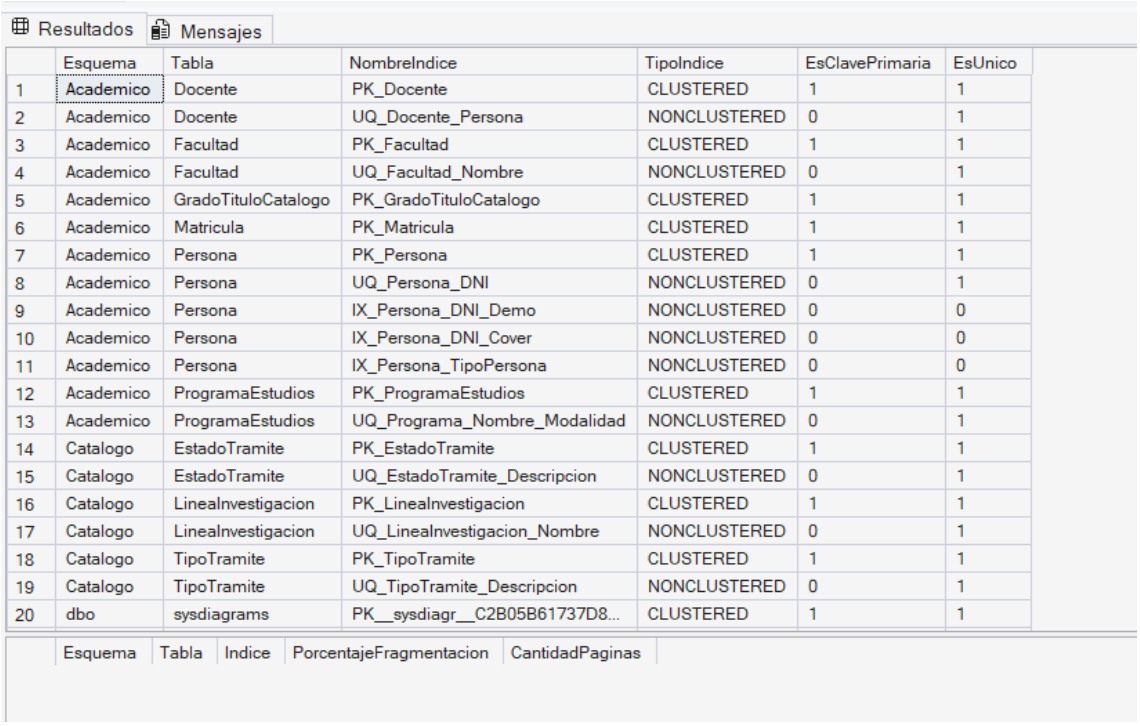
AUDITORÍA, MONITOREO Y ANÁLISIS DE FRAGMENTACIÓN DE ÍNDICES




```sql
USE GradosTitulos;
GO

-- 1. Auditoría general de todos los índices creados por esquema y tabla
SELECT
    s.name AS Esquema,
    t.name AS Tabla,
    i.name AS NombreIndice,
    i.type_desc AS TipoIndice,
    i.is_primary_key AS EsClavePrimaria,
    i.is_unique AS EsUnico
FROM sys.indexes i
INNER JOIN sys.tables t ON i.object_id = t.object_id
INNER JOIN sys.schemas s ON t.schema_id = s.schema_id
WHERE i.name IS NOT NULL
ORDER BY s.name, t.name, i.type;
GO

-- 2. Análisis de fragmentación física de los índices en el sistema
SELECT
    OBJECT_SCHEMA_NAME(ps.object_id) AS Esquema,
    OBJECT_NAME(ps.object_id) AS Tabla,
    i.name AS Indice,
    ps.avg_fragmentation_in_percent AS PorcentajeFragmentacion,
    ps.page_count AS CantidadPaginas
FROM sys.dm_db_index_physical_stats(DB_ID(), NULL, NULL, NULL, 'LIMITED') ps
INNER JOIN sys.indexes i ON ps.object_id = i.object_id AND ps.index_id = i.index_
WHERE ps.avg_fragmentation_in_percent > 10
ORDER BY ps.avg_fragmentation_in_percent DESC;
GO
```



Resultados | Mensajes

	Esquema	Tabla	NombreIndice	TipoIndice	EsClavePrimaria	EsUnico
1	Academico	Docente	PK_Docente	CLUSTERED	1	1
2	Academico	Docente	UQ_Docente_Persona	NONCLUSTERED	0	1
3	Academico	Facultad	PK_Facultad	CLUSTERED	1	1
4	Academico	Facultad	UQ_Facultad_Nombre	NONCLUSTERED	0	1
5	Academico	GradoTituloCatalogo	PK_GradoTituloCatalogo	CLUSTERED	1	1
6	Academico	Matricula	PK_Matricula	CLUSTERED	1	1
7	Academico	Persona	PK_Persona	CLUSTERED	1	1
8	Academico	Persona	UQ_Persona_DNI	NONCLUSTERED	0	1
9	Academico	Persona	IX_Persona_DNI_Demo	NONCLUSTERED	0	0
10	Academico	Persona	IX_Persona_DNI_Cover	NONCLUSTERED	0	0
11	Academico	Persona	IX_Persona_TipoPersona	NONCLUSTERED	0	0
12	Academico	ProgramaEstudios	PK_ProgramaEstudios	CLUSTERED	1	1
13	Academico	ProgramaEstudios	UQ_Programa_Nombre_Modalidad	NONCLUSTERED	0	1
14	Catalogo	EstadoTramite	PK_EstadoTramite	CLUSTERED	1	1
15	Catalogo	EstadoTramite	UQ_EstadoTramite_Descripcion	NONCLUSTERED	0	1
16	Catalogo	LineaInvestigacion	PK_LineaInvestigacion	CLUSTERED	1	1
17	Catalogo	LineaInvestigacion	UQ_LineaInvestigacion_Nombre	NONCLUSTERED	0	1
18	Catalogo	TipoTramite	PK_TipoTramite	CLUSTERED	1	1
19	Catalogo	TipoTramite	UQ_TipoTramite_Descripcion	NONCLUSTERED	0	1
20	dbo	sysdiagrams	PK__sysdiagr__C2B05B61737D8...	CLUSTERED	1	1

Esquema	Tabla	Indice	PorcentajeFragmentacion	CantidadPaginas

4) Cuestionario de Cierre de Laboratorio

¿Cuál es el umbral de fragmentación recomendado en SQL Server a partir del cual se debe evaluar la reconstrucción (REBUILD) frente a una simple reorganización (REORGANIZE) de un índice?

Respuesta Teórica: Conforme las tablas reciben operaciones intensivas de inserción, actualización y borrado (DML), los índices sufren fragmentación interna. En SQL Server, tras monitorear el porcentaje mediante sys.dm_db_index_physical_stats, el umbral estándar recomendado por Microsoft establece que:

1. **Entre el 10% y el 30% de fragmentación:** Se debe aplicar una **reorganización (REORGANIZE)**, ya que es una operación liviana que desfragmenta únicamente el nivel hoja del índice reutilizando las páginas existentes sin exigir un alto costo de CPU ni bloqueos prolongados.
2. **A partir de un umbral superior al 30% (> 30%) de fragmentación:** Se debe evaluar y ejecutar la **reconstrucción completa (REBUILD)**. A este nivel, una simple reorganización ya no es eficiente; el REBUILD crea una estructura de índice completamente nueva en disco, compacta las páginas de datos y actualiza las estadísticas de distribución, devolviendo el rendimiento óptimo a las consultas. (Como consideración adicional en

entornos reales, estas operaciones suelen aplicarse sobre índices que tengan más de 1,000 páginas en page_count para que el impacto en el rendimiento sea significativo).